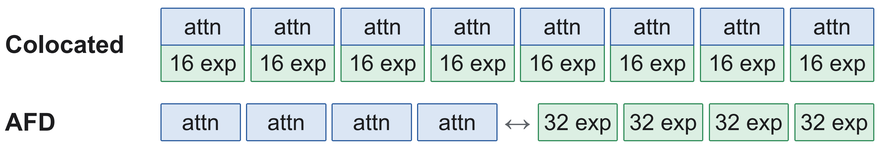
Improving the LLM serving Pareto frontier with AFD on TPUs

Charlie Xue, Akhil Chilaka, Rohan Anne, Jerry Ji

Motivation

- In a mixture-of-experts (MoE) model, attention's cost grows with the number and length of requests, since it works over each request's KV cache.
- The experts hold most of the weights and are shared by all requests, but each token uses only a few, so they run efficiently only when many tokens are batched together.
- Colocated serving runs both on every chip and must scale them together, while attention–FFN disaggregation (AFD) gives each its own chips.
- On GPUs, ByteDance's MegaScale-Infer uses AFD to get up to 1.9× higher throughput per GPU, and we are testing whether AFD works on TPUs.



Why TPUs make this difficult

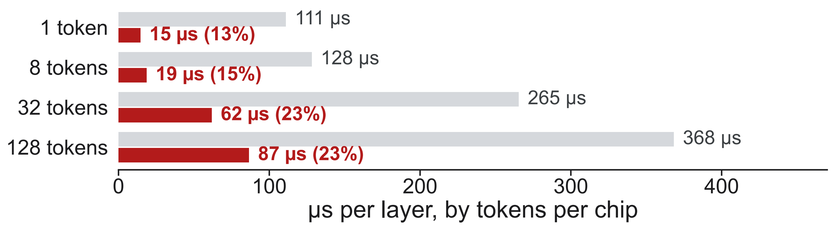
torch_tpu, the PyTorch backend that vLLM uses on TPUs, compiles each step into one program that every chip runs, and chips exchange data only through collectives that all of them call together. AFD needs the two groups to do different work, and our first attempts failed:

- Giving each group its own program made the collectives they share hang.
- torch_tpu doesn't support a runtime branch (`torch.cond`) that would let one program choose a role.
- An all-to-all that sends different amounts to each chip, as AFD's exchange needs, doesn't compile.
- Sending tokens chip to chip outside the compiled step works, but takes 1.3 ms per layer, about 90 times slower.

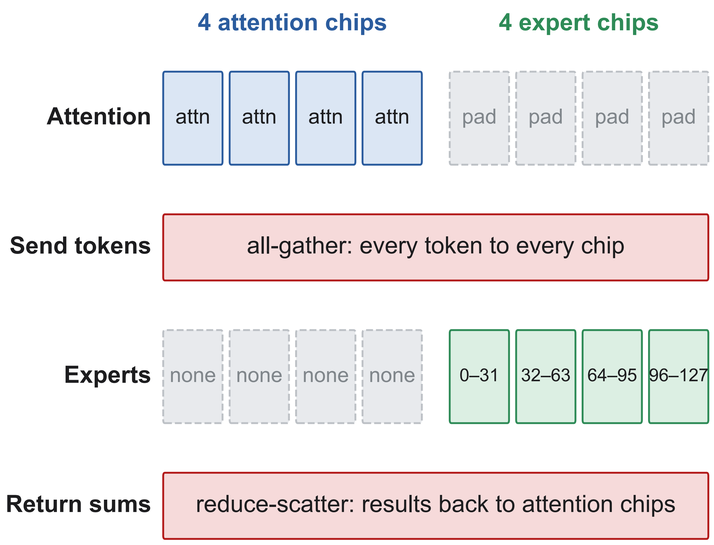
We ended up running the same program on every chip and passing each chip's role in as data.

Cost of the token exchange

In every layer, AFD sends each token from the attention chips to the expert chips and brings the results back. With the shared program, this takes 13 to 23% of the time of a colocated layer (**red** vs grey). That overhead is small enough for AFD to be worth pursuing.



Our design



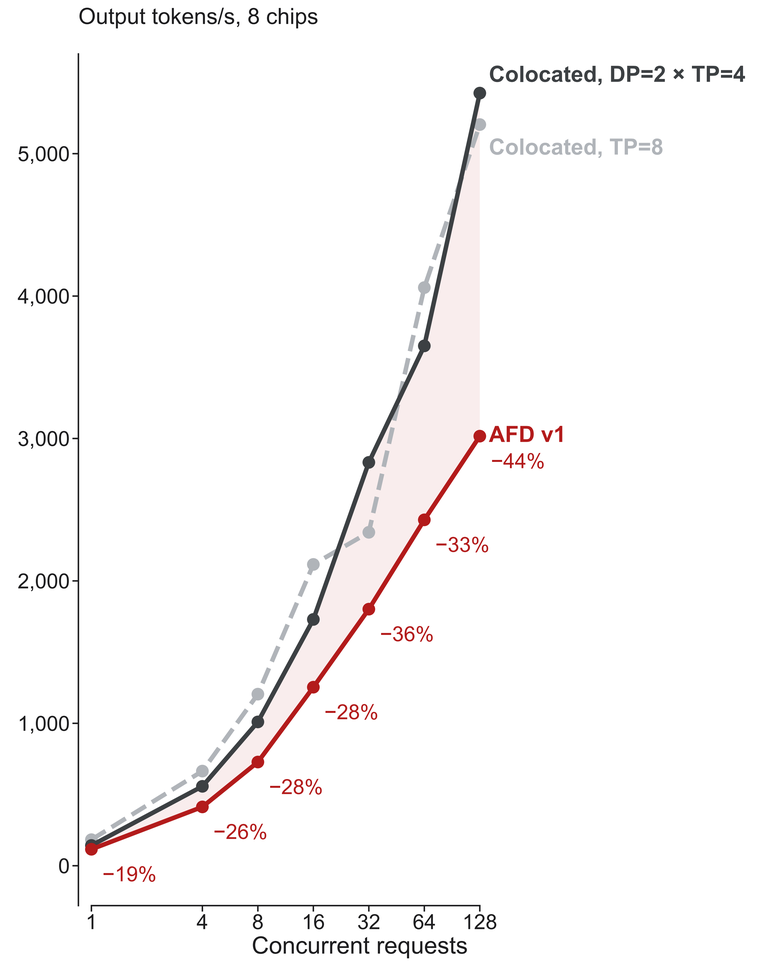
- We serve Qwen3-30B-A3B, an MoE model with 128 experts and 58 GB of expert weights, on 8 TPU v6e chips with vLLM.
- vLLM runs two copies of the model on 4 chips each (DP=2 × TP=4), and both copies execute the same compiled program in lockstep.
- We send every request to the first copy and give its chips an empty expert range, so they skip the expert work and act as attention chips, though they still load expert weights.
- The second copy gets no requests, so its chips act as expert chips, each holding 32 of the 128 experts.
- Tokens reach the expert chips and come back through vLLM's existing all-gather and reduce-scatter, shown as the red rows.
- The grey cells are wasted work, since the expert chips run attention on padding and the attention chips sit idle while the experts run.

Checking correctness

AFD answers all 4 of our test prompts correctly, such as "What is 17 × 23?" With greedy decoding, its output also matches colocated serving word for word until the model sees two almost equally likely next words, where tiny bf16 rounding differences can flip the choice:

Prompt	AFD	Colocated
"In 1969, the Apollo 11 mission"	**marked** (0.239)	**achieved** (0.239)
"The three laws of thermodynamics are the"	**foundation** (0.233)	**most** (0.206)

Our results



Against DP=2 × TP=4, the gap grows from 19% at 1 request to 44% at 128. At 128 requests, AFD also takes twice as long to the first token (2.0 s vs 1.0 s) and gives each user 34 tokens/s instead of 61. We think attention is the bottleneck, since AFD runs all of it on 4 chips instead of 8.

Next steps

- Profile one layer of AFD and of colocated serving to measure what each grey cell costs, and fix the biggest bottleneck first.
- Split each batch in half so attention works on one half while the experts work on the other, as MegaScale-Infer does on GPUs.
- Remove the padding work on the expert chips and the unused expert weights on the attention chips, freeing memory for more requests.
- Test giving attention more chips than the experts, since attention limits AFD at high load.